

임우진 · Lim Woojin

웹 포트폴리오에서 좀 더 편하게 확인하실 수 있습니다.

[웹 포트폴리오 보기 ↗](#)

네트워크를 좋아하는 클라우드/DevOps 엔지니어 임우진 입니다.

사용자의 요청이 네트워크와 서버를 지나 데이터베이스까지 처리되는 전체 흐름을 이해하고 설계하는 것을 좋아합니다. 계층별로 나누어진 시스템의 흐름을 따라가며, 서비스가 end-to-end로 연결되는 구조를 이해하는 것이 중요하다고 생각합니다.

여러 기술 분야를 이해하고 연결하는 개발자를 지향합니다.

프론트엔드, 백엔드, 인프라, 네트워크, 보안, 임베디드, AI, 운영체제 등 다양한 분야를 공부하고 프로젝트를 진행해왔습니다. 하나의 기술만 사용하는 개발자가 아니라, 서비스의 전체 분야를 이해하고 필요한 기술을 적재적소에 사용할 수 있는 개발자를 지향합니다.

불편함을 해결하고 자동화하기 위해 개발합니다.

DevOps는 개발과 운영 사이에서 반복되는 배포, 검증, 모니터링 흐름을 정리하고 자동화해 서비스가 안정적으로 전달되도록 만드는 역할이라고 생각합니다. 복잡한 워크플로우를 단순화하고, 사람이 직접 확인하던 작업을 재현 가능한 시스템 흐름으로 바꾸는 데 관심이 있습니다.

연락처

전화번호	010-2296-1280	이메일	woojin2296@gmail.com
깃허브	https://github.com/woojin2296	기술 블로그	velog.io/@talking_tomato

기술 스택

- Cloud / Infra AWS, Terraform, Kubernetes, Linux, Nginx
- CI/CD / GitOps GitHub Actions, Argo CD, Git, Docker Compose
- Observability Prometheus, Grafana, Loki, Alloy
- Backend Spring Boot, Java, MySQL, Python
- Frontend Next.js, TypeScript

도봉라이프(DobongLife)

서울시 도봉구 지역상권 활성화 애플리케이션의 AWS 인프라를 구축하고 운영한 프로젝트

PERIOD

2026.04-current

POSITION

(주)유머스트알앤디 / 사업부 IT팀

ROLE

DevOps Engineer / Cloud Engineer

프로젝트 개요

도봉라이프 애플리케이션의 **AWS 인프라 아키텍처**를 설계하고 구축했으며, **배포 파이프라인**과 **모니터링 환경**을 구성해 운영하고 있습니다. 도봉라이프는 서울특별시 도봉구의 지역상권 활성화 애플리케이션으로 스토리텔링 기반 여행 코스, 맛집 정보, 지역 소상공인 연계 할인 쿠폰 발행 및 사용 기능을 제공하며, 현재 **플레이스토어**와 **앱스토어**에 출시되어 다운로드 가능합니다.

역할

- 단일 EC2 기반 아키텍처(V1)를 역할 기반 AWS 인프라(V2)와 **EKS 기반 MSA 플랫폼**(V3)으로 확장 설계
- Terraform 기반 IaC**를 적용하여 AWS 인프라를 코드로 관리하고, 스택 분리를 통해 재현성과 변경 추적성 확보
- GitHub Actions, OIDC, SSM, Argo CD 기반 GitOps **CI/CD 파이프라인** 구축
- AWS Secrets Manager를 활용한 **Secret 관리** 및 주입 체계 구축
- HPA, Auto Scaling**을 활용한 AWS 인프라의 확장성 및 가용성 확보
- Grafana, Prometheus를 활용한 **모니터링 시스템** 구축
- TLS 인증서 갱신 실패 **운영 장애 대응**

기술 스택

AWS

EKS / Kubernetes

Terraform

GitHub Actions

Argo CD

AWS Secrets Manager

Prometheus

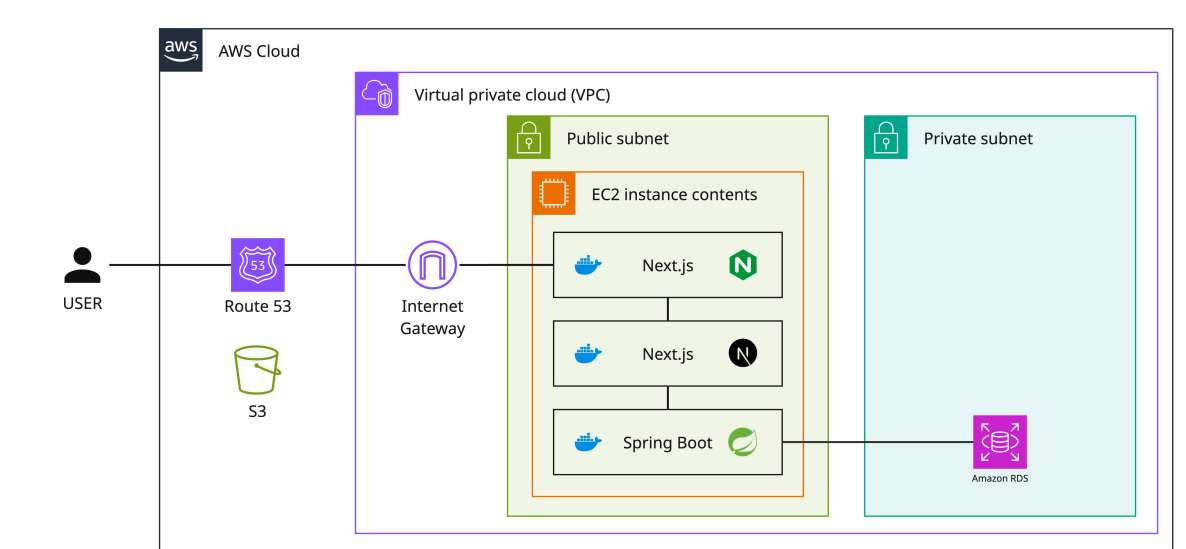
Grafana

Loki

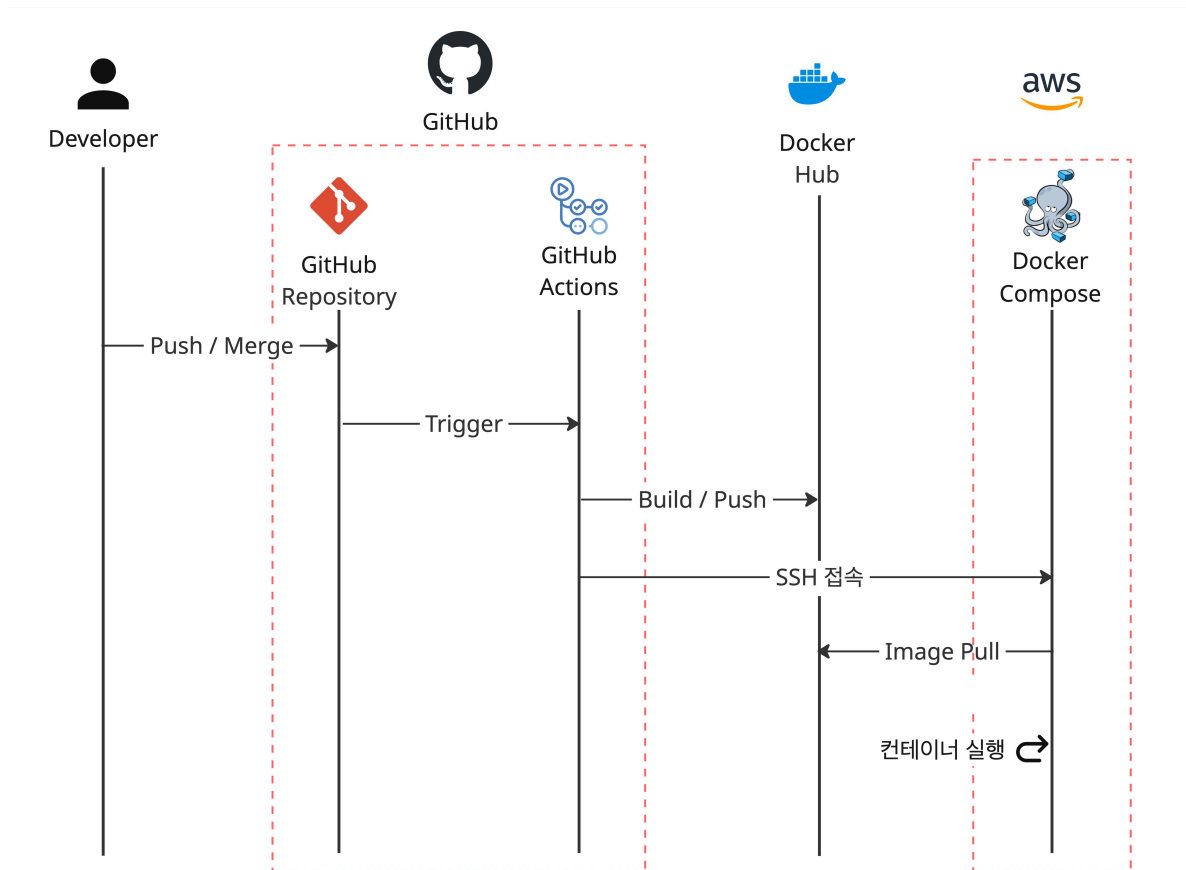
AWS 인프라 및 CI/CD 방식 개선 (V1, V2, V3)

기존 레거시 인프라 구조(V1)

기존 인프라는 단일 EC2에서 Nginx, Spring Boot, Redis를 함께 운영하는 구조였습니다. 배포 파이프라인의 경우 GitHub Actions에서 Docker 이미지를 빌드한 뒤 **Docker Hub**에 업로드하고, EC2에 **SSH**로 접속해 이미지를 내려받아 컨테이너를 갱신하는 방식이었습니다.



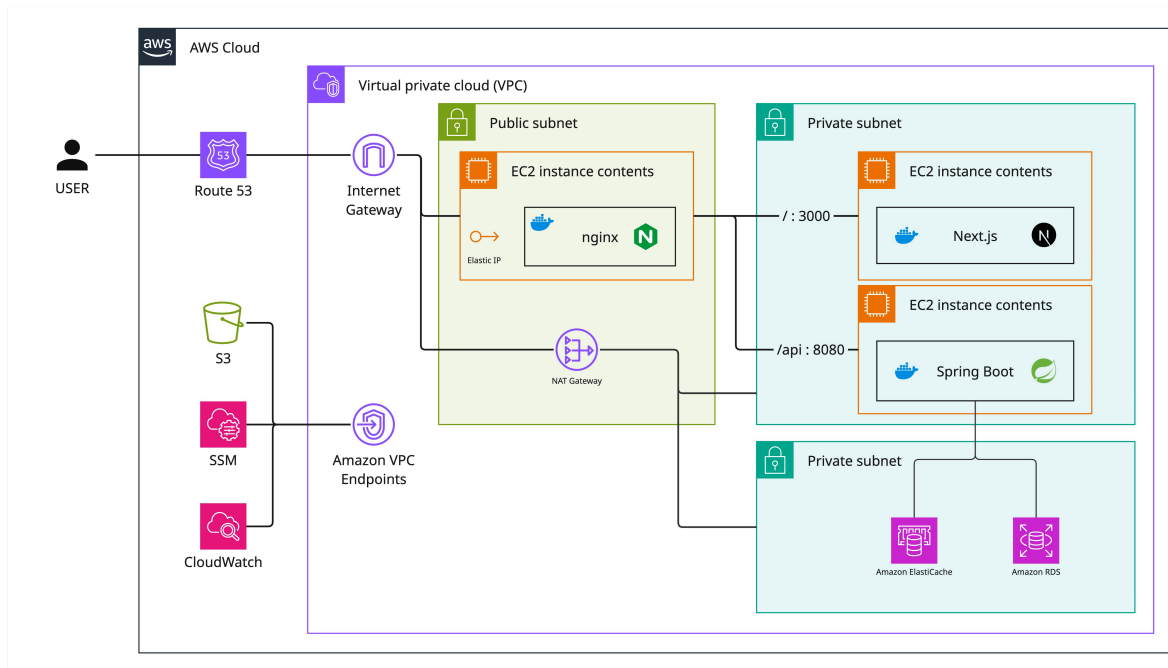
도봉라이프 AWS 인프라 구조 다이어그램 V1



도봉라이프 AWS 인프라 구조 다이어그램 V1

역할 기반 인프라 구조로 전환(V2)

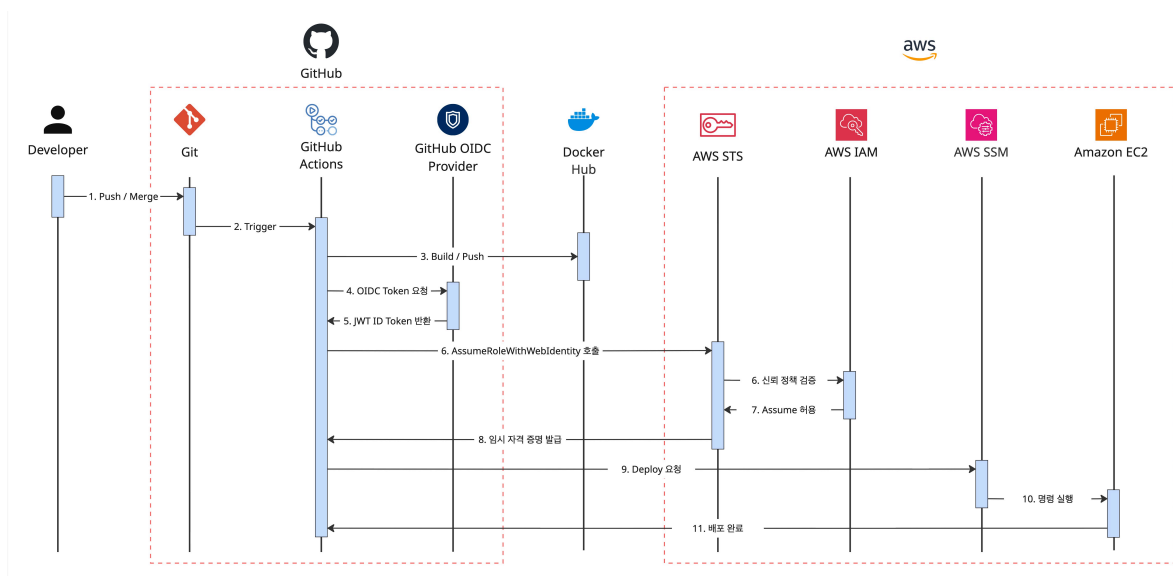
프론트 서버를 별도로 배포해야 해 기존 인프라를 역할 기반 아키텍처로 분리하고 확장하였습니다. 이에 따라 Nginx, Backend, Frontend 서버를 **역할별 EC2로 분리**하고, 외부 접근 영역과 내부 서비스 영역을 Public/Private Subnet으로 나누었습니다. Redis는 ElastiCache로 분리해 애플리케이션 서버와 상태 저장 계층의 책임을 나누었고, Amazon CloudWatch를 활용해 EC2 로그 수집 및 애플리케이션 로그 확인 환경을 구축했습니다.



도봉라이프 AWS 인프라 구조 다이어그램 V2

GitHub Actions, OIDC, SSM을 활용한 CI/CD 파이프라인 구축 (V2)

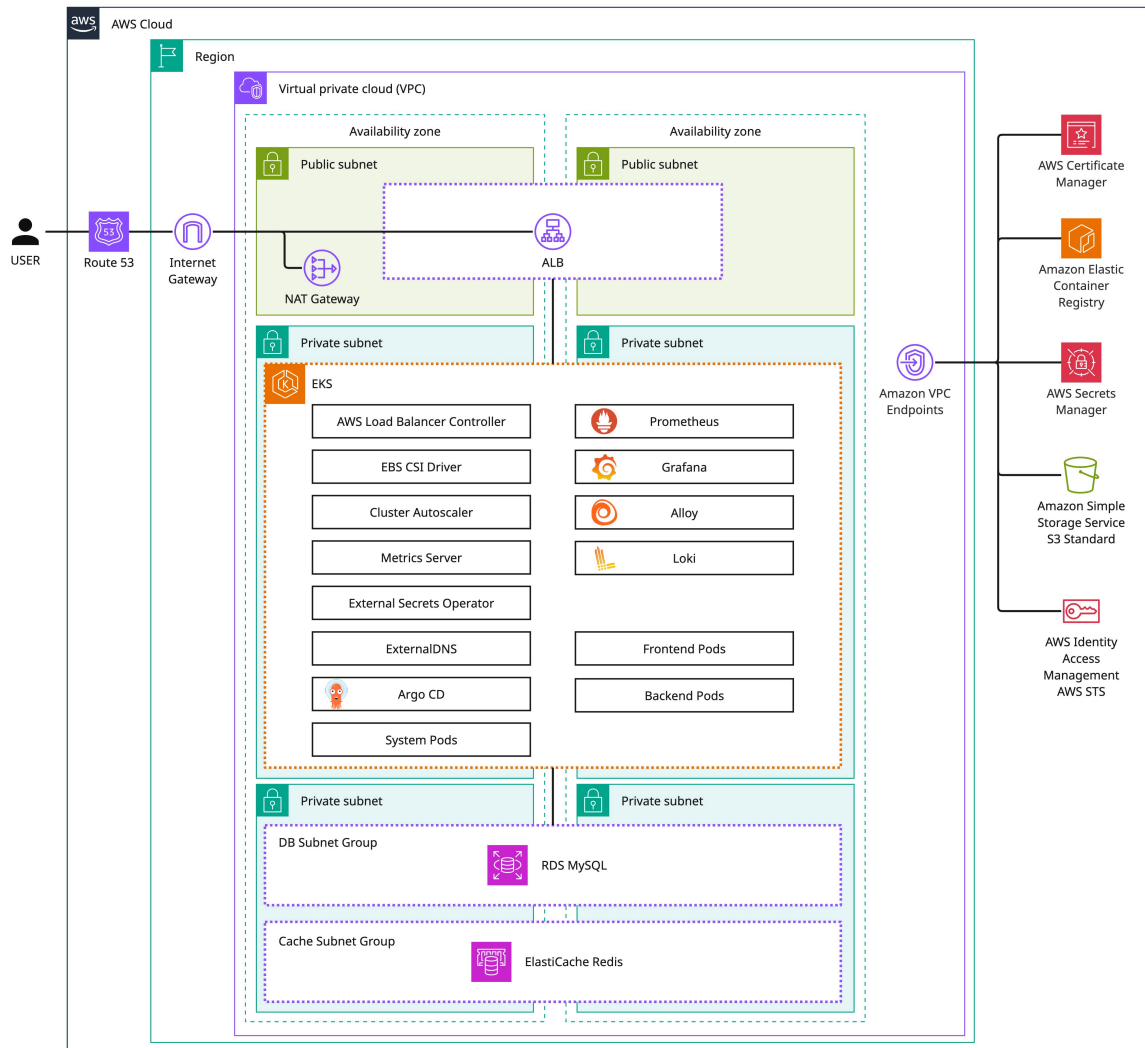
서버에 직접 SSH로 접속해 배포하고 환경 변수를 관리하던 방식을 줄이고, 기존 SSH 22번 포트 기반 접근을 제거했습니다. **AWS Systems Manager**로 인스턴스 접근 방식을 전환하고, GitHub Actions와 **OIDC** 기반 권한 위임을 사용해 CD 파이프라인을 재 구성했습니다.



도봉라이프 CI/CD 파이프라인 구조 다이어그램 V2

마이크로 서비스 전환을 위한 EKS 환경 구축 (V3)

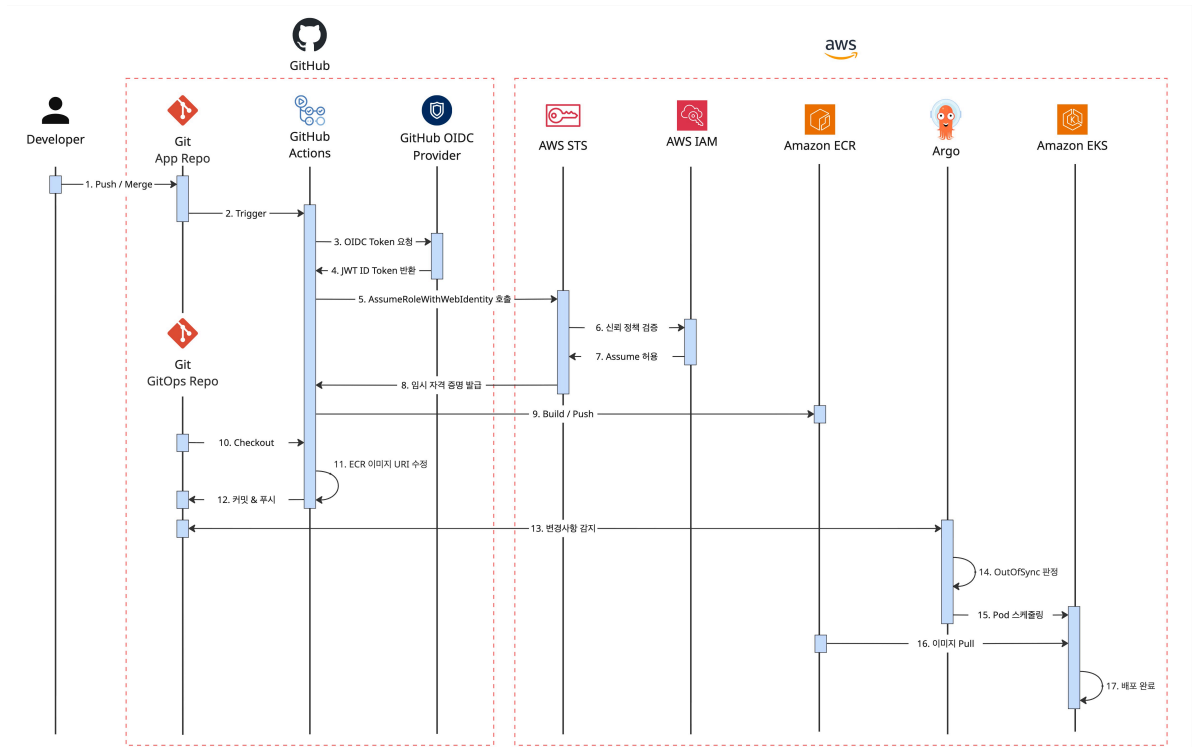
백엔드 서비스의 MSA 아키텍처 전환과 트래픽 증가에 대비해 기존 서버 배포 방식을 **AWS EKS 기반 구조**로 확장했습니다. 전체 구조는 외부 진입, 애플리케이션 실행, 데이터 저장 계층을 분리하는 방식으로 설계했습니다. Public Subnet은 ALB를 통한 외부 요청 진입점으로 두고, Private Subnet 내부에는 EKS 애플리케이션 노드와 RDS/Redis 데이터 계층을 나누어 배치해 서비스 트래픽과 데이터 접근 경계를 분리했습니다.



도봉라이프 AWS 인프라 구조 다이어그램 V3

Argo CD와 GitOps 기반 EKS 배포 파이프라인 구축 (V3)

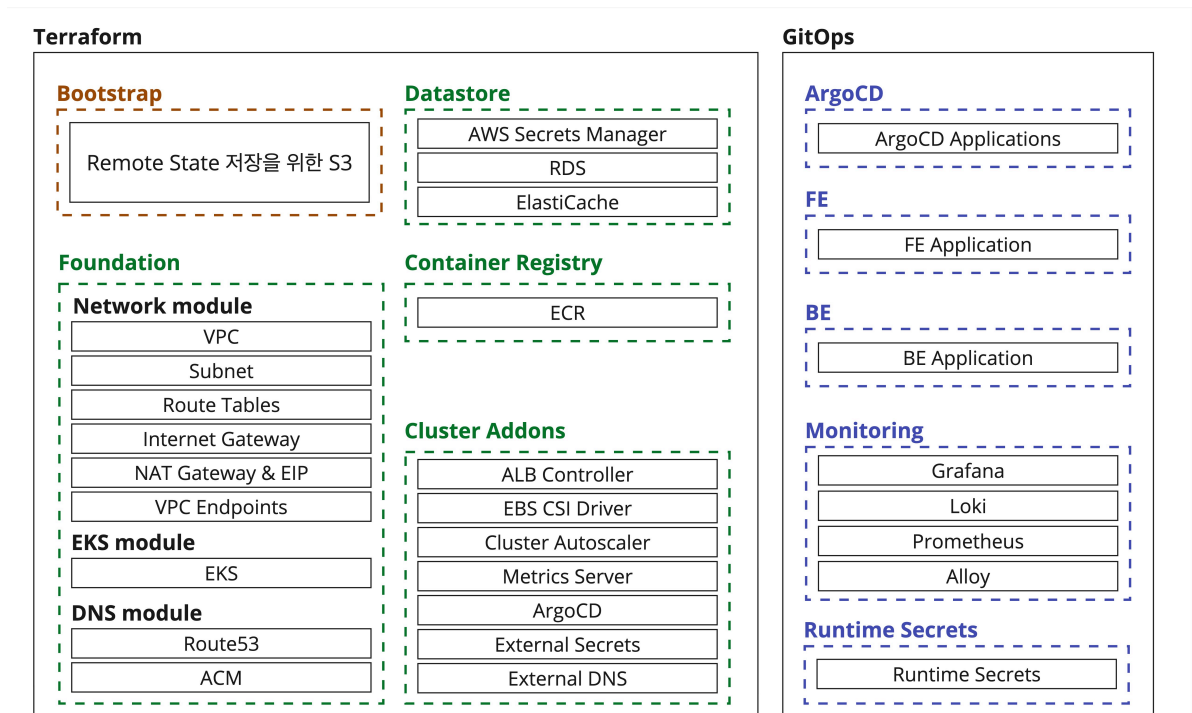
Argo CD를 사용해 이미지 빌드와 클러스터 배포 책임을 분리하는 방식으로 구성했습니다. **GitHub Actions**는 서비스 소스 변경 시 **Docker** 이미지를 빌드해 **Amazon ECR**에 업로드하고, **GitOps** 저장소의 **Helm values** 이미지 태그만 갱신하도록 했습니다. **GitHub webhook**은 **GitOps** 저장소 변경 이벤트를 **Argo CD**로 전달해 변경 감지 지연을 줄이도록 연결했으며, 이후 **Argo CD**에서 **GitOps** 저장소 변경을 기준으로 **EKS**에 동기화했습니다.



도봉라이프 CI/CD 파이프라인 구조 다이어그램 V3

Terraform 기반 IaC와 GitOps 운영 구조 구축

Terraform 기반 IaC로 AWS 인프라를 코드화하여 인프라 재현성과 변경 추적성을 높였습니다. Terraform은 인프라 리소스의 목표 상태를 정의하고 생성·변경을 적용하고, 리소스 성격과 변경 범위에 따라 스택을 분리했습니다. **GitOps 저장소**는 Kubernetes 애플리케이션의 배포 상태를 관리하고, **Argo CD**는 해당 변경을 감지해 EKS 클러스터에 동기화하도록 했습니다.



도봉라이프 Terraform과 GitOps 운영 구조 다이어그램

Secret 저장 위치와 주입 흐름 분리

민감값은 Terraform 코드나 GitOps 저장소에 직접 두지 않고 **AWS Secrets Manager**에 저장해, 저장 위치와 참조 방식을 분리했습니다. 인프라 계층은 secret 값 자체가 아니라 필요한 참조 정보만 전달하고, 애플리케이션은 **Kubernetes Secret**으로 동기화된 값을 환경 변수로 사용하도록 구성했습니다.

가용성 및 자동 확장 구성

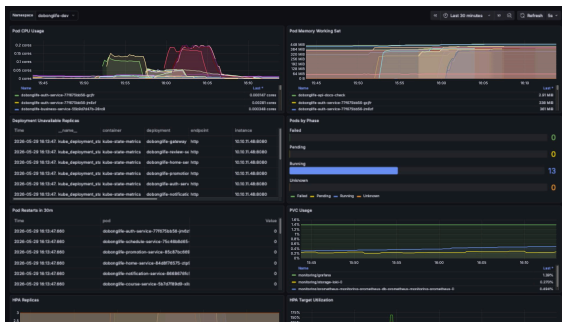
서비스 부하와 Pod 배치 상태에 대응할 수 있도록 **HPA**와 **Cluster Autoscaler**를 함께 구성했습니다. HPA는 서비스별 부하에 따라 Pod replica 수를 조정하고, Cluster Autoscaler는 Pending Pod가 발생했을 때 Managed Node Group을 확장할 수 있도록 구성했습니다.

모니터링 구성

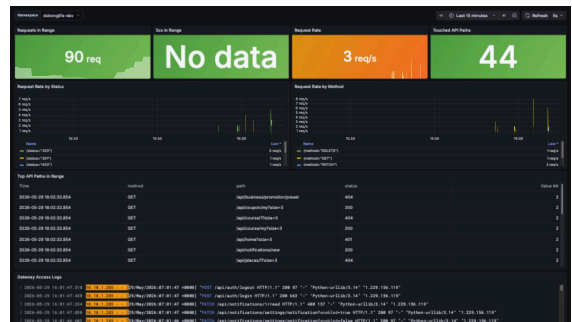
Grafana와 **Prometheus**, **Loki**를 활용해 EKS 클러스터와 애플리케이션의 모니터링 시스템을 구축했습니다.

메트릭 흐름은 Prometheus가 클러스터와 워크로드 지표를 수집하고, Grafana에서 Pod 상태, 리소스 사용량, HPA 동작을 확인하는 구조로 구성했습니다. API 지표는 요청 수, 5xx 발생량, request rate, method/status/path별 호출량을 중심으로 확인하도록 정리했습니다.

로그 흐름은 Grafana Alloy가 워크로드와 API 로그를 수집하고 Loki에 저장한 뒤, Grafana에서 메트릭과 로그를 함께 조회하는 방식으로 구성했습니다. 이를 통해 Pod 상태나 API 오류가 발생했을 때 관련 로그를 같은 관측 화면에서 함께 확인할 수 있도록 했습니다.



도봉라이프 Grafana 운영 대시보드



도봉라이프 API 트래픽과 오류 로그 대시보드

V1 환경 API 서버 운영 장애 대응: TLS 인증서 갱신 실패

장애 상황

프론트엔드 동료로부터 앱 로그인에 되지 않는다는 제보를 받고 V1 환경의 API 서버 장애를 확인했습니다. 확인 결과 일반 로그인, 카카오 로그인, 비밀번호 찾기, 이메일 인증처럼 **API 호출이 필요한 인증 기능 전반이 실패**했고, 앱에는 네트워크 연결 상태가 원활하지 않습니다 오류가 표시되었습니다.

원인 확인

먼저 서버에 접속해 애플리케이션 프로세스를 확인했습니다. 새로운 요청을 시도했지만, 백엔드 로그와 Nginx access log에 신규 접속 로그가 남지 않았습니다. 이를 통해 **요청이 애플리케이션까지 도달하지 못한다는 것을 확인**하였습니다.

초기에 DNS 문제를 의심하여 도메인 접속과 서버 IP 직접 접속을 비교하려 했지만, 도메인 접속 단계에서 **인증서 만료 오류가 확인**되었습니다. 이후 **openssl 명령어**로 인증서 정보를 확인했고, 인증서 만료로 장애가 발생했다는 것을 확인하였습니다.

조치 및 재발 방지

이후 인증서 갱신을 위해 **certbot renew**를 시도하였으나, Nginx가 이미 80번 포트를 점유하고 있어 실패했습니다. 긴급 복구를 위해 Nginx를 일시 중단한 뒤 인증서를 갱신했고, 갱신 후 Certbot 상태와 외부에서 제공되는 인증서 만료일을 다시 확인했습니다. 재발 방지를 위해 기존 renewal 설정의 authenticator 항목을 **standalone**에서 **webroot**로 변경했습니다.

프로젝트 회고

사수 없이 인프라 전반을 단독으로 구축하며 부족한 부분과 시행착오도 많았지만, 그만큼 직접 부딪히며 많이 성장할 수 있었던 프로젝트입니다. 서비스 전체를 AWS 환경에 직접 올리고 배포하면서, 인프라가 어떻게 연결되고 실제 트래픽이 어떤 흐름으로 처리되는지 이해할 수 있었습니다. 특히 도메인 요청이 어떤 라우팅을 거쳐 서버까지 도달하는지, 포트 설정에 따라 요청 흐름이 어떻게 달라지는지 직접 확인하는 과정이 가장 재밌었습니다. 또한 Terraform을 사용해보면서 기존에 GUI를 통해 설정했을 때에는 알지 못했던 인프라의 전체 구조와 흐름을 더 쉽게 이해할 수 있었습니다.

판옵티콘(Panopticon)

연구실 실험 데이터 수집 파이프라인 관제 및 이상 알림 시스템을 개발한 프로젝트

PERIOD

2024.07-2026.02 (1년 8개월 - 개발 및 운영)

POSITION

순천향대학교 UBICOMP LAB / 학부연구생

ROLE

Team Lead / Full Stack Developer

프로젝트 개요

순천향대학교 UBICOMP LAB의 연구실 실험 데이터 수집 파이프라인을 위한 **실시간 관제 및 이상 알림 시스템**입니다. 기존 환경에서는 수집이 멈추거나 데이터 품질에 문제가 생겨도 직접 데이터를 열람하기 전까지 이상 여부를 확인하기 어려웠고, 실제로 **데이터 수집 중단을 3일간 인지하지 못한 사례**가 있었습니다. 이 문제를 해결하기 위해 서버 응답 상태, 마지막 수집 시각, 데이터 품질을 기준으로 장애 여부를 자동 감지하도록 했습니다. 감지된 상태는 통합 대시보드와 실시간 알림을 통해 확인할 수 있도록 구성했습니다. 개발 이후 졸업 전까지 약 **1년 8개월**간 연구 과제 환경에서 실제 운영했으며, 현재도 연구실 실험 데이터 수집 환경에서 계속 운영되고 있습니다.

역할

- 팀 리드로서 요구사항 정리, 기능 범위 정의, 전체 시스템 아키텍처 설계
- 데이터 수집·조화·장애 상태 흐름을 고려한 DB 스키마 설계 및 **Spring Boot** API 구현
- 장치 상태, 최근 수집 시각, 센서 데이터를 한 화면에서 확인하는 관제 UI 설계 및 **Next.js** 프론트엔드 구현
- self-hosted runner** 기반 CI/CD 파이프라인 구축 및 Docker 기반 배포 환경 구성

기술 스택

Next.js

Spring Boot

MySQL

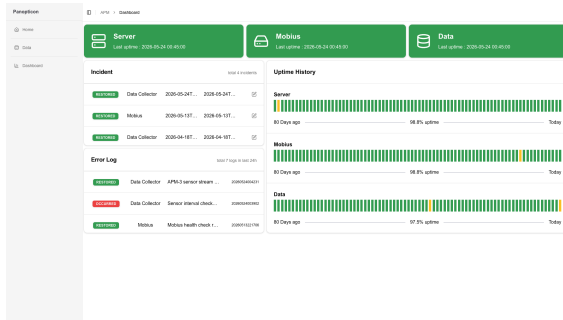
Docker Compose

GitHub Actions

Self-hosted Runner

데이터 수집 파이프라인 관제 시스템 개발

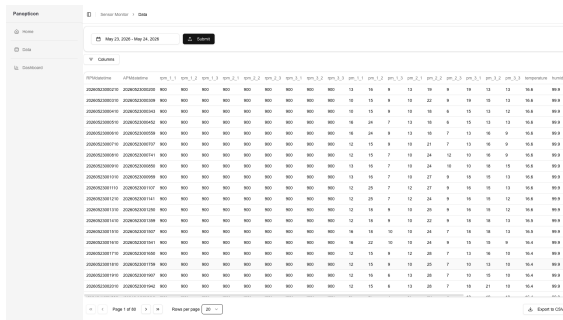
서버와 데이터베이스의 응답 상태, 센서 데이터 수집 상태, 장애 이력을 대시보드에서 통합적으로 확인할 수 있도록 구성했습니다. 데이터 확인 과정도 DB 직접 접근에 의존하지 않도록 바꿨습니다. 운영자는 화면에서 실시간 데이터를 확인하고, 필요한 기간을 선택해 데이터를 조회한 뒤 CSV로 출력할 수 있습니다. 감지된 이상 상태는 Spring Boot 백엔드에 저장하고, 대시보드에 반영하는 동시에 이상 발생 시 Slack으로 알리를 전송하도록 연결했습니다.



리소스 상태와 장애 이력을 확인하는 대시보드 화면



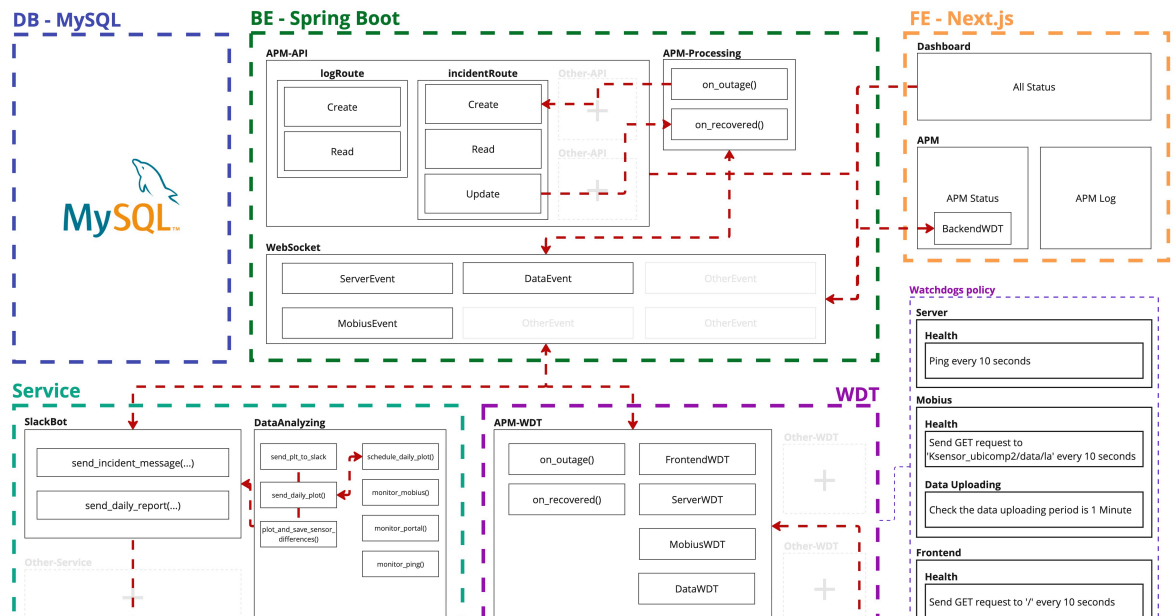
실시간 데이터 수집 상태를 확인하는 관제 대시보드 화면



기간별 데이터 조회와 CSV 출력을 위한 상세 화면

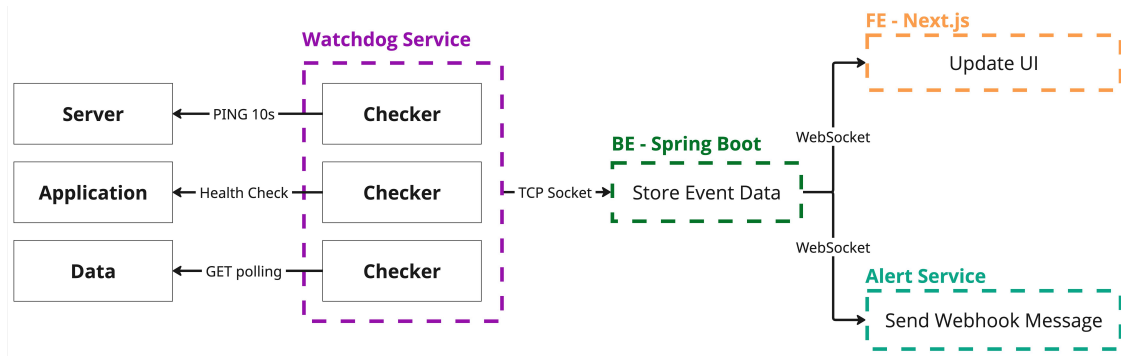
확장성을 고려한 모듈 아키텍처 설계

Watchdog의 감지 로직과 Notification의 알림 전송 로직을 분리했습니다. 감지 대상이 늘어나더라도 해당 대상의 **checker** 모듈을 추가하는 방식으로 쉽게 확장할 수 있도록 구조를 잡았습니다. Slack 외의 알림 채널이 필요하면 **notifier** 모듈을 추가하는 구조로 설계했습니다. 백엔드는 Watchdog이 보낸 감지 결과를 저장하고, 프론트엔드는 저장된 상태 이벤트를 조회하거나 **WebSocket**으로 전달받아 화면을 갱신합니다.



TCP Socket과 WebSocket을 사용한 상태 이벤트 전달 구조 구현

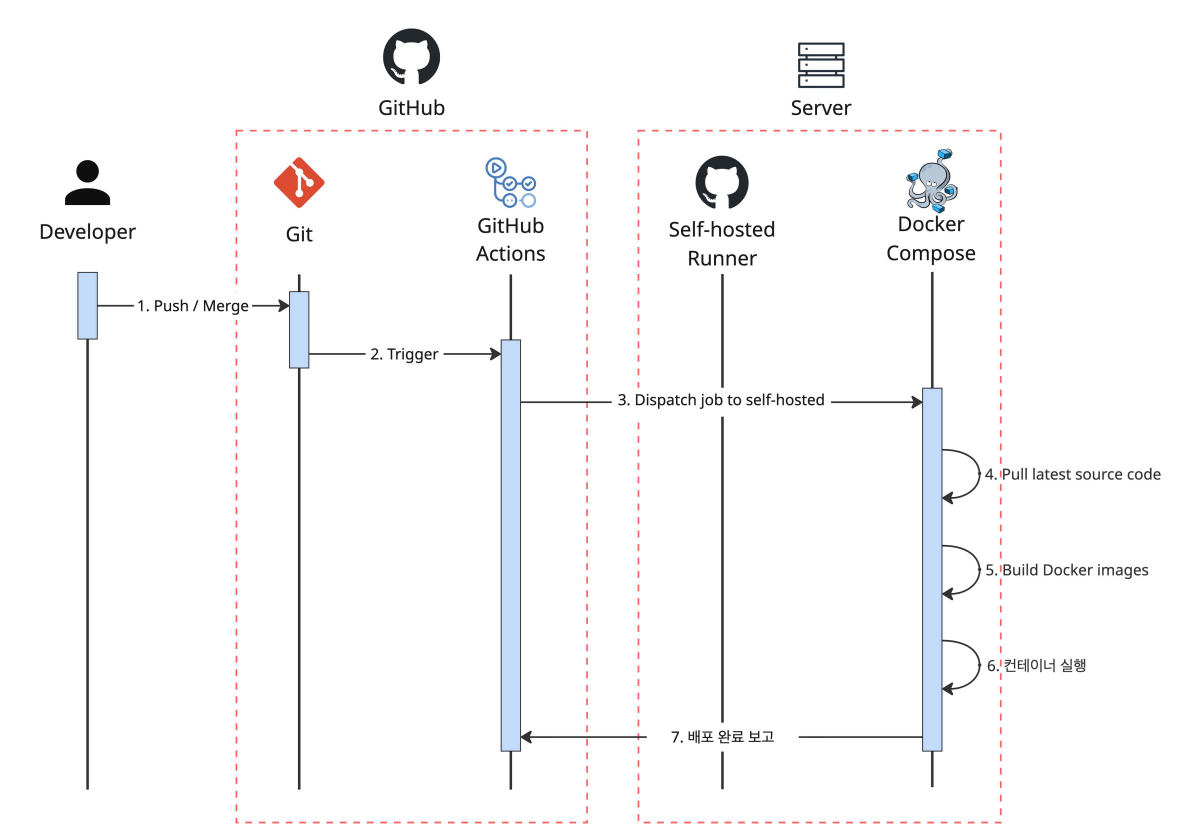
Watchdog은 각 감지 대상에 맞는 checker 전략을 실행하고 이상 상태를 판단합니다. 이상 상태가 확인되면 Watchdog은 감지 결과를 TCP 기반 소켓 통신으로 Spring Boot 백엔드에 전달합니다. 백엔드는 감지 결과를 상태 이벤트로 저장한 뒤 **WebSocket** 이벤트로 변환해 Next.js 프론트엔드에 전달합니다. Slack 알림도 같은 상태 이벤트를 기준으로 전송되도록 연동했습니다.



Watchdog 상태 이벤트 전파 구조

Self-hosted Runner 기반 내부망 배포 자동화 구축

서버가 학교 내부망에 있어 외부에서 SSH로 접속하는 배포 파이프라인을 사용할 수 없었습니다. 내부망 서버에 **self-hosted runner**를 설치하고, GitHub Actions job이 해당 runner에서 실행되도록 구성했습니다. GitHub Actions는 내부망 runner에 job을 전달하고, runner가 서버 안에서 소스 갱신, Docker 이미지 빌드, Docker Compose 기반 컨테이너 재배포를 순서대로 수행하도록 구성했습니다.



self-hosted runner를 사용한 CI/CD 시퀀스 다이어그램

회고 및 개선 방향

시스템 구축 이후 사용률이 높아 보람찼던 프로젝트였습니다. 특히 실시간 데이터 그래프와 알림 시스템이 연구실 운영 과정에서 유용하게 사용되었습니다. 운영하면서 감지 대상과 알림 채널을 분리해 둔 점도 유지보수에 도움이 됐습니다. 새 서버를 감시하거나 Slack 외의 알림 방식을 붙일 때 Watchdog 전체를 수정하지 않고 모듈을 추가하는 방식으로 대응할 수 있었습니다. 반대로 Watchdog과 백엔드 사이를 TCP Socket으로 직접 연결한 구조는 네트워크 하위 계층을 직접 다루는 방식이라 신경 써야 할 부분이 많았습니다. 기존에 만들어진 코드가 있어 그대로 사용했지만, 다시 설계한다면 Watchdog의 감지 결과 전달은 HTTP/Webhook 기반 방식을 사용하거나, 프론트엔드 단방향 상태 갱신도 WebSocket 대신 SSE 등의 기술을 사용할 것 같습니다.

SCH MiniProject PMS

순천향대학교 사물인터넷학과 ML/DL 강의에서 사용하는 프로젝트 과제 관리 시스템을 개발한 프로젝트

PERIOD

2025.06-2026.02 (10개월 - 개발 및 운영)

POSITION

순천향대학교 ML/DL 강의 / 개인 프로젝트

ROLE

Full Stack Developer

프로젝트 개요

순천향대학교 사물인터넷학과 머신러닝·딥러닝 강의에서 사용하는 **프로젝트 과제 관리 시스템**입니다. 기존 방식에서는 학생이 서로의 점수를 비교할 수 없어 자신의 결과 수준을 확인하기 어려웠고, 교수자는 제출 파일과 점수를 따로 확인해 순위를 직접 계산해야 했습니다. 이를 해결하기 위해 학생은 결과를 제출하면 자신의 랭킹을 확인할 수 있고, 교수자는 학생 점수와 제출 파일을 웹에서 관리할 수 있는 시스템을 구현했습니다. 실제 강의 성적에 반영되는 데이터를 다루기 때문에 점수 위변조를 막는 서버 측 검증을 포함해 세션 기반 사용자 관리, 감사 로그, 파일 업로드 검증을 구현했습니다. 이 시스템은 졸업 전까지 **10개월간** 직접 운영했으며, 현재도 강의 운영 환경에서 계속 사용되고 있습니다. 현재 **월 평균 약 40명의 학생**이 과제 제출과 랭킹 조회 기능을 이용하고 있습니다.

역할

- 교수자 요구사항을 바탕으로 과제 제출, 점수 비교, 랭킹, 관리자 기능의 요구사항 정리
- 학생용 과제 제출/랭킹 화면과 교수자용 관리 화면의 UI/UX 설계 및 **프론트엔드 구현**
- 서버 세션 인증, 역할 기반 접근 제어, 서버 측 점수 검증, 파일 업로드 검증, 감사 로그 개발
- Docker Compose** 기반 실행 환경 구성과 졸업 전까지 배포 및 운영 대응

기술 스택

Next.js

SQLite

Docker Compose

[illegible][illegible]

아키텍처 및 데이터 구조 설계

The diagram illustrates the architecture of a Docker Compose application. It is divided into two main sections: **Docker Compose** (outlined in orange) and **Server Volume** (outlined in red).

Docker Compose Section:

- Next.js:** A green dashed box containing:
 - Pages / UI:**
 - 학생용 화면 (Student Interface): 1. 과제 제출 (Assignment Submission), 2. 랭킹 조회 (Ranking Inquiry)
 - 관리자용 화면 (Admin Interface): 1. 유저 관리 (User Management), 2. 점수 관리 (Score Management), 3. 요청 로그 관리 (Request Log Management)
 - API:**
 - 인증 API (Authentication API)
 - 제출 API (Submission API)
 - 랭킹 API (Ranking API)
 - 요청 로그 API (Request Log API)
- DB - SQLite:** A box containing the SQLite logo and text.
- 업로드 파일 저장소 (Upload File Storage):** A box containing an icon of three stacked disks.

Server Volume Section:

- DB Volume:** A box connected to the DB - SQLite component.
- Upload Volume:** A box connected to the 업로드 파일 저장소 component.

Flow and Connections:

- A **USER** (represented by a person icon) interacts with the **Next.js** application.
- There is a bidirectional arrow between the **Pages / UI** and **API** components.
- The **API** component connects to both the **DB - SQLite** and the **업로드 파일 저장소**.
- The **DB - SQLite** component connects to the **DB Volume**.
- The **업로드 파일 저장소** component connects to the **Upload Volume**.

```

    erDiagram
        users ||--o{ users : "1:M"
        users ||--o{ scores : "1:M"
        users ||--o{ sessions : "1:M"
        users ||--o{ request_logs : "1:M"
        users ||--o{ notices : "1:M"
        scores ||--o{ sessions : "1:M"
        sessions ||--o{ request_logs : "1:M"
        sessions ||--o{ notices : "1:M"
  
```

users

id	id	Domain	BIGINT
student_number	student_number	Domain	VARCHAR(50)
email	email	Domain	VARCHAR(100)
password_hash	password_hash	Domain	VARCHAR(255)
name	name	Domain	VARCHAR(100)
public_id	public_id	Domain	VARCHAR(64)
role	role	Domain	VARCHAR(20)
is_active	is_active	Domain	BOOLEAN
last_login_at	last_login_at	Domain	DATETIME
created_at	created_at	Domain	DATETIME
updated_at	updated_at	Domain	DATETIME

scores

id	id	Domain	BIGINT
user_id	user_id	Domain	BIGINT
project_number	project_number	Domain	INT
score	score	Domain	DECIMAL(10,2)
file_path	file_path	Domain	VARCHAR(500)
file_name	file_name	Domain	VARCHAR(255)
file_type	file_type	Domain	VARCHAR(20)
file_size	file_size	Domain	BIGINT
created_at	created_at	Domain	DATETIME
updated_at	updated_at	Domain	DATETIME

sessions

id	id	Domain	BIGINT
user_id	user_id	Domain	BIGINT
session_token	session_token	Domain	VARCHAR(255)
expires_at	expires_at	Domain	DATETIME
created_at	created_at	Domain	DATETIME

request_logs

id	id	Domain	BIGINT
source	source	Domain	VARCHAR(50)
path	path	Domain	VARCHAR(100)
method	method	Domain	VARCHAR(20)
status	status	Domain	INT
ip_address	ip_address	Domain	VARCHAR(64)
metadata	metadata	Domain	JSON
created_at	created_at	Domain	DATETIME

notices

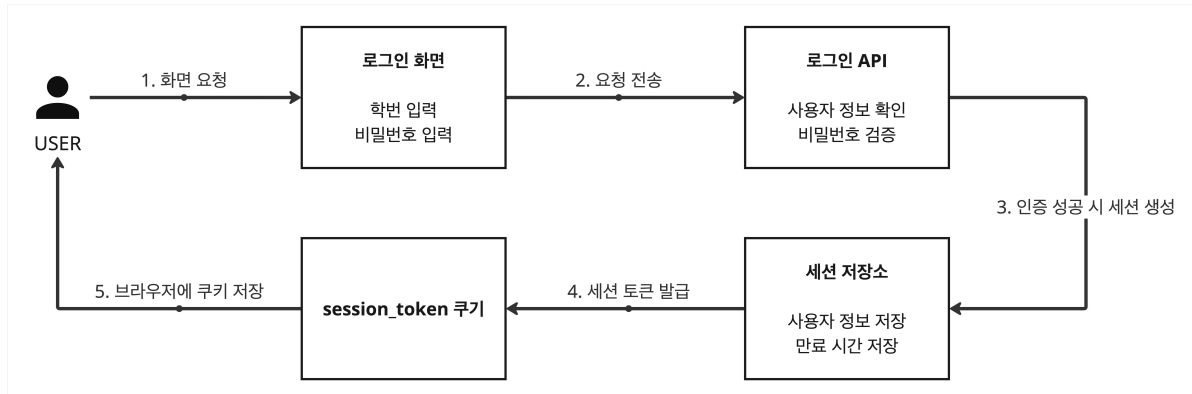
id	id	Domain	BIGINT
message	message	Domain	TEXT
is_active	is_active	Domain	BOOLEAN
created_at	created_at	Domain	DATETIME
updated_at	updated_at	Domain	DATETIME

데이터 구조 ERD

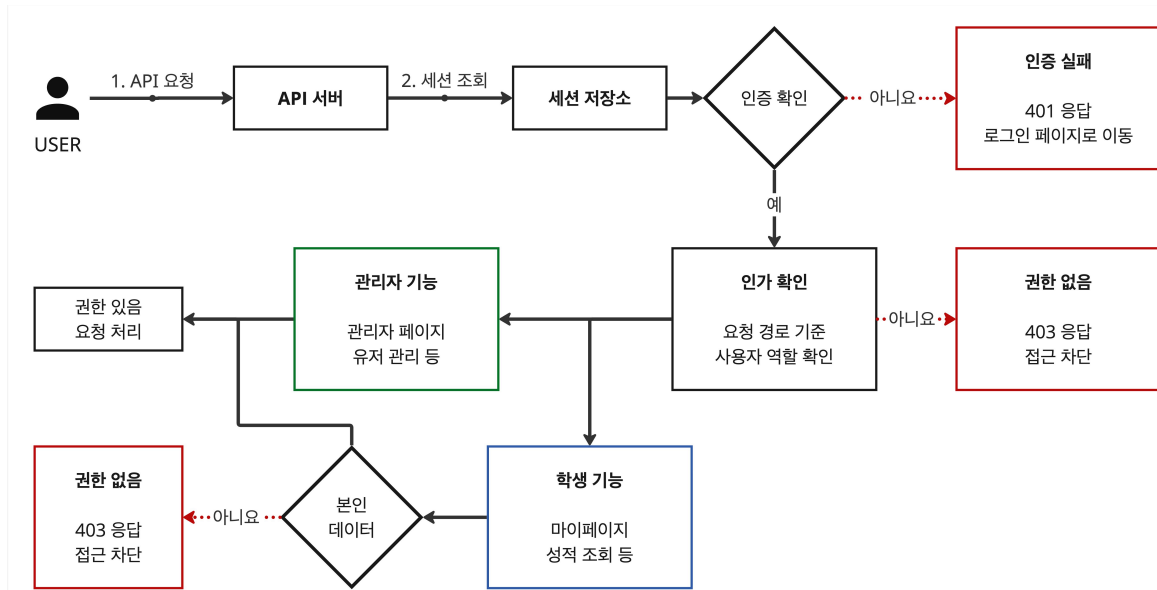
세션 기반 인증과 API 경로 기반 인가 구현

강의 운영 시스템은 외부 서비스 연동보다 학생·관리자 권한을 서버에서 확실하게 통제하는 것이 중요했습니다. 그래서 토큰을 클라이언트에서 직접 다루는 방식보다 서버가 로그인 상태와 권한을 관리하는 **세션 기반 인증**을 선택했습니다. 인증은 학번 기반 로그인과 session_token 쿠키, 서버 세션을 기준으로 처리했습니다.

보호 경로에서는 세션을 먼저 확인하고, 학생용 API와 관리자용 API를 경로 기준으로 분리해 역할에 맞는 요청만 처리되도록 했습니다. 학생은 본인의 제출 결과와 랭킹 조회 기능에 접근하고, 관리자는 사용자 관리, 점수 집계, 제출 파일 확인 같은 운영 기능에 접근할 수 있도록 인가 범위를 나눴습니다.



로그인 및 세션 발급 흐름



보호된 API 접근 및 경로 기반 인가 흐름

파일 업로드 보안과 서버 검증 구현

제출 파일은 성적 처리의 근거가 되기 때문에 단순 첨부 파일이 아니라 검증 대상 데이터로 다뤘습니다. 입력 단계, 저장 단계, 다운로드 단계에서 필요한 확인을 나눠 적용했습니다.

서버 배포 장애 대응: 내부망 DNS 설정 유실

장애 상황

배포 과정에서 Docker Compose가 필요한 이미지를 pull하는 단계에서 실패했습니다. 로그 확인 결과 IP 주소 대상 ping은 성공했지만 도메인 이름으로는 ping이 실패했습니다.

원인 확인

초기에 DNS 문제를 의심하여 서버의 netplan 설정에서 nameservers.addresses 값을 Cloudflare DNS(1.1.1.1)와 Google DNS(8.8.8.8)로 지정해 공개 DNS로 변경해 확인했습니다. 하지만 도메인 이름으로는 ping이 계속 실패했습니다.

이후 원인을 다시 확인하면서, 학교 내부망에서는 외부 공개 DNS로 직접 질의하지 못하고 학교 로컬 DNS 서버를 통해 도메인을 해석해야 하는 구조일 가능성이 높다고 판단했습니다.

조치 및 재발 방지

유실되어 있던 학교 DNS 서버 IP를 서버 DNS 설정에 반영했고, 도메인 해석과 Docker Compose 배포가 정상 진행되는 것을 확인했습니다.

회고 및 개선 방향

UI 디자인을 AI를 활용하여 진행했는데, 생각보다 원하는 분위기에 가깝게 나와 제작 과정이 즐거웠던 프로젝트였습니다. 성적에 반영되는 프로젝트이다 보니 보안을 최대한 신경 쓰려고 노력했지만 추후 검토 과정에서 놓친 부분들이 많았다는 것을 알게 되었습니다. 특히 파일 업로드 보안에서 단순한 확장자 검증과 구조 검증뿐만 아니라 MIME type, 파일 signature까지 확인해야 한다는 것을 알게 되어 후임자에게 해당 내용을 공유하고, 향후 시스템을 새로 구축할 때는 이러한 부분들을 보완해서 시스템을 설계해야겠다고 생각했습니다.

경험

(주)유머스트알앤디 | 계약직 사원 | 2026.04 ~ Current
도봉라이프 AWS 인프라 구축 및 운영 · 회사 ERP 시스템 백엔드 개발 및 운영

(주)유머스트알앤디 | 인턴 사원 | 2025.09 ~ 2026.02 · 6개월
X-ray 치료기기 GUI 개발 참여 · 사내 환경 관제 시스템 개발

순천향대학교 UBICOMP LAB | 학부연구생 | 2024.01 — 2026.02 · 2년
데이터 수집 파이프라인 관제 시스템 개발 - 판옵티콘 · ROS 기반 PCD 수집 파이프라인 구축 · KCC2025 학부생 논문 제출
수업 조교 6회 | 순천향대학교 · 2024.01 — 2025.12
IoT 플랫폼 (24-1, 25-1) · 임베디드 시스템 (24-2, 25-2) · 나만의 게임만들기 (24-1) · 웹페이지 제작의 실제 (24-2)
해커톤 참여 3회 | 기획 · 개발 · 발표 · 2024.01 — 2025.12
2024 캡스톤디자인 및 AI 해커톤 · 2025 캡스톤디자인 및 AI 해커톤 · 2025 대한민국 해커톤
멘토링 프로그램 2회 | 멘토 · 2024.01 — 2025.12
2024 순천향 AI·SW 창의한마당 · 2025 SW 학습멘토링(머신러닝)

수상 내역

2024.11 · 최우수상	SW융합대학 학술제 (E-Sports 개발) 캐주얼 리듬게임 - 탭 스페이스 (Tab Space)
2025.11 · 장려상	2025 SCHU AI SW Festival AI 기반 발표 연습 보조 서비스 - 플라보 (PLAVO)
2025.11 · 장려상	2025 캡스톤 디자인 및 AI 해커톤 장소에 맞는 음악 생성 서비스 - 카라멜 (Caramel)
2025.08 · 장려상	2025 글로벌 캡스톤디자인 경진대회 AI 기반 발표 연습 보조 서비스 - 플라보 (PLAVO)
2024.10 · 장려상	2024 캡스톤디자인 및 AI 해커톤 입법 현황 실시간 분석 시스템 - 라다온 (LawDaon)

프로필

학력	순천향대학교 사물인터넷학과 졸업예정 2020.03 ~ 2026.08 · 평점 4.22/4.5	어학	영어 : TOEIC 810 2025.11 취득
병역	군필 : 육군 병장 만기 전역 (MW 통신병) 2022.05 ~ 2023.11	논문	한국컴퓨터종합학술대회 학부생 논문 발표 논문 확인하기 ↗

Other Projects

Image Denoising Project - 이미지 노이즈 제거 필터 비교 분석 모듈

[GitHub ↗](#)

개인 프로젝트 · 2023.11 ~ 2023.12

Salt & Pepper Noise와 Gaussian Noise가 포함된 이미지에 여러 필터링 기법을 적용하고, PSNR 기준으로 결과 품질을 비교한 이미지 처리 프로젝트입니다.

- Salt & Pepper, Gaussian Noise 기반 테스트 데이터 생성
- Median Filter, Non-Local Means, Hybrid 필터링 알고리즘 구현
- PSNR 지표 기반 필터별 이미지 품질 비교 및 실험 결과 분석

[Python](#)[NumPy](#)[OpenCV](#)[Matplotlib](#)

Mobius Subscription - MQTT 구독 처리 파이썬 모듈

[GitHub ↗](#)

Ubicomp Lab K-Sensor 프로젝트 · 2024.10

Mobius에서 발생하는 IoT 데이터 이벤트를 실시간으로 수집하기 위한 Python 기반 MQTT 구독 모듈입니다.

- Mobius REST API 기반 AE-컨테이너 구독 생성 자동화 구현
- MQTT topic별 이벤트 수신과 JSON payload 파싱 로직 구현
- 멀티 스레드 기반 다중 구독 처리와 종료 정리 구조 설계

[Python](#)[MQTT](#)[oneM2M](#)[Mobius](#)

플라보 (PLAVO) - AI 기반 발표 연습 보조 서비스

[GitHub ↗](#)

순천향대학교 사물인터넷학과 졸업작품 · 2024.09 ~ 2025.06

AI와 IoT 기술을 결합해 사용자의 발표를 실시간으로 분석하고, 연습 기록과 피드백 리포트를 제공하는 발표 연습 보조 서비스입니다.

- Next.js 기반 발표 연습 화면, 결과 리포트, 성장 그래프 UI 구현
- 음성·자세 분석 결과를 실시간으로 반영하는 WebSocket 처리 흐름 설계
- 프로젝트·연습 기록·분석 결과 관리를 위한 MySQL 데이터 모델링과 API 구현

[Next.js](#)[MySQL](#)[Raspberry Pi](#)

루미온 (LUMION) - 대화형 AI 자동 지식 아카이빙 서비스

2025 대한민국 해커톤 · 2025.11

대화형 AI 사용 과정에서 생성되는 정보를 자동으로 정리하고, 마인드맵 형태로 구조화해 탐색할 수 있도록 만든 지식 아카이빙 서비스입니다.

- 사용자 입력, 답변 생성, 문장 임베딩, 유사도 계산, 그래프 구성으로 이어지는 AI 처리 흐름 설계
- 대화 데이터를 노드·엣지 구조로 변환하는 마인드맵 기반 시각화 UI 구현
- 노드 클릭 기반 확장 탐색과 지식 관계를 단계적으로 보여주는 인터랙션 설계

Next.js

카라멜 (Caramel) - 장소에 맞는 AI 음악 생성 서비스

2025 캡스톤디자인 및 AI 해커톤 경진대회 · 2025.11

GitHub ↗

사용자의 사진과 상황 입력을 기반으로 장소에 어울리는 음악을 생성하고, 생성 결과를 바로 재생할 수 있도록 만든 AI 기반 음악 생성 서비스입니다.

- 사용자 입력, 프롬프트 생성, 음악 생성, 결과 재생으로 이어지는 프론트엔드 흐름 설계
- Gemini API와 Suno API를 연결한 AI 음악 생성 요청·응답 처리 구현
- 음악 생성 중 로딩 상태와 생성 완료 후 플레이어 UI를 포함한 인터랙션 구현

Next.js

Gemini API

Suno API

라다온 (Lawdaon) - 입법 현황 실시간 분석 시스템

2024 캡스톤디자인 및 AI 해커톤 경진대회 · 2024.10

GitHub ↗

입법 발의 및 처리 현황 데이터를 수집·분석해 최신 입법 동향과 통계 정보를 제공하는 실시간 분석 시스템입니다.

- 입법 데이터 수집 및 처리 로직 구현
- 입법 현황 조회와 분석 결과 제공을 위한 Spring Boot API 설계
- 데이터 수집, 분석, 제공으로 이어지는 백엔드 처리 흐름 구현

Spring Boot

Tab Space - Unity 기반 캐주얼 리듬게임

순천향대학교 제1회 SW중심대학 학술제 e-sports 공모전 · 2024.10

음악 타이밍에 맞춰 스페이스바를 입력해 방향 전환과 점프를 수행하고, 점수와 랭킹으로 경쟁하는 Unity 기반 캐주얼 리듬게임입니다.

- 스페이스바 입력 기반 이동, 방향 전환, 점프 플레이 로직 구현
- 정확도 판정과 추가 점수 계산을 포함한 리듬게임 점수 시스템 구현
- 목숨 판정, 게임 진행 UI, 랭킹보드 반영 흐름 구현

Unity

C#

소방 보조용 스마트 헬멧 - IMU 기반 실내 상대위치 추적 및 관제 시스템

[GitHub ↗](#)

IoT 플랫폼 기말 프로젝트 · 2024.05 ~ 2024.06

GPS가 동작하기 어려운 실내 환경에서 IMU 센서 데이터를 활용해 소방관의 상대 위치를 추정하고, 중앙에서 실시간으로 관제하는 시스템입니다.

- 가속도·자이로 데이터를 활용한 PDR 기반 상대 위치 계산 로직 구현
- Mobius oneM2M 플랫폼을 활용한 센서 데이터 수집 흐름 설계
- Three.js 기반 3D 위치 시각화와 Spring Boot 위치 데이터 API 구현

Spring Boot

MySQL

Python

Mobius

oneM2M

Three.js

Vue.js

Raspberry Pi

Smart Bus Stop - 임베디드 시스템 기반 스마트 버스정류장 시스템

[GitHub ↗](#)

순천향대학교 사물인터넷학과 임베디드 시스템 기말 프로젝트 · 2021.09 ~ 2021.12

버스 도착 정보와 센서 기반 환경 정보를 디스플레이로 제공하고, 사용자 입력 기반 인터랙션을 지원하는 스마트 버스정류장 시스템입니다.

- Arduino 기반 센서 데이터 수집 및 처리 로직 구현
- 디스플레이 기반 버스 도착 정보와 환경 정보 표시 UI 구현
- 하드웨어와 소프트웨어 연동 구조 설계 및 통합 테스트 수행

Arduino

C

Embedded System